```
$ homelabx --report
```

HomeLabX

Technical report: architecture, isolation, the browser terminal and how to run it on your own machine

React + TypeScript | FastAPI | PostgreSQL | Redis | Docker | xterm.js | noVNC | Playwright | GitHub Actions

A self-hosted cyber-security training platform with isolated, disposable labs, 13 AppSec scenarios and a browser-based Linux AttackBox (terminal or desktop).

Version 0.1.0 · Repository: github.com/ParitV/HomeLab

Executive summary

HomeLabX is a personal, TryHackMe-style training platform that runs entirely on one machine. A learner picks a lab, presses *Launch*, and within seconds gets a set of freshly created containers on a private network: an **AttackBox** (a Linux workstation with security tooling) and one or more **deliberately vulnerable targets**. The AttackBox is used through a real Linux terminal embedded in the web page. The learner exploits the target, captures a *flag* (a secret string that proves success), submits it, and the platform records progress. *Reset* throws the whole environment away and builds a clean one with a new flag.

Because the targets are intentionally insecure, the project's central engineering problem is **containment**: the vulnerable machines must be fully exploitable, while the platform around them, the host computer and the local network stay protected. Every design decision in this report follows from that.

13	~250	6	2
labs across web, API, auth and DevSecOps, each with its own scenario and flag	automated tests across unit, security, Docker integration and browser E2E	CI jobs that must all pass (incl. security audits and a full-stack E2E run)	AttackBox types: a terminal (xterm.js) and a full XFCE desktop (noVNC)

Contents

1 What HomeLabX does
2 Architecture at a glance
3 Technology stack
4 What "the host" is: Docker, containers, the Linux VM
5 From lab definition to running lab
6 Networking and isolation
7 How the browser terminal works
8 The first lab: Broken Customer Portal
9 Flags, objectives and progress
10 Security model
11 Testing and continuous integration
12 Packaging: running it on another machine
13 Roadmap
A Repository map and glossary

Key design decisions in one paragraph each

- **Docker as the lab engine.** Each lab instance is a set of containers on its own Docker network. Containers start in seconds, are cheap to throw away (which makes *reset* trivial) and provide Linux isolation primitives (namespaces, cgroups, capabilities).
- **One isolated network per lab, with no way out.** Lab networks are *internal* bridges without a gateway: no internet, no LAN, no route to the host or to other labs, and no route to the platform's own services.
- **The web platform never gives lab containers Docker access.** Only the backend talks to Docker, and only through a filtering proxy that blocks dangerous API calls such as `exec`.
- **The terminal uses Docker *attach*, not *exec* and not SSH.** The AttackBox's shell runs on a TTY; the backend attaches to it through the Docker API and relays keystrokes and output over a WebSocket. No network path from a lab into the platform is ever needed. A second AttackBox flavour, a full XFCE desktop, is reached the same way: the backend relays its noVNC WebSocket over an internal-only gateway network, so no VNC port is ever published.

- **Exploit, then fix, then verify.** At least one lab per category goes beyond "find the flag": the learner edits the vulnerable source file in the browser, the platform rebuilds the target with that fix, and a hardened verifier re-runs the original exploit plus a regression check before the objective completes.
- **Security is tested, not just documented.** Every requirement in the security specification maps to an automated test, and CI fails on any regression.

1 What HomeLabX does

The learner journey is deliberately short: **Dashboard → Lab → Launch → AttackBox → exploit → submit flag → objective complete → reset or stop**.

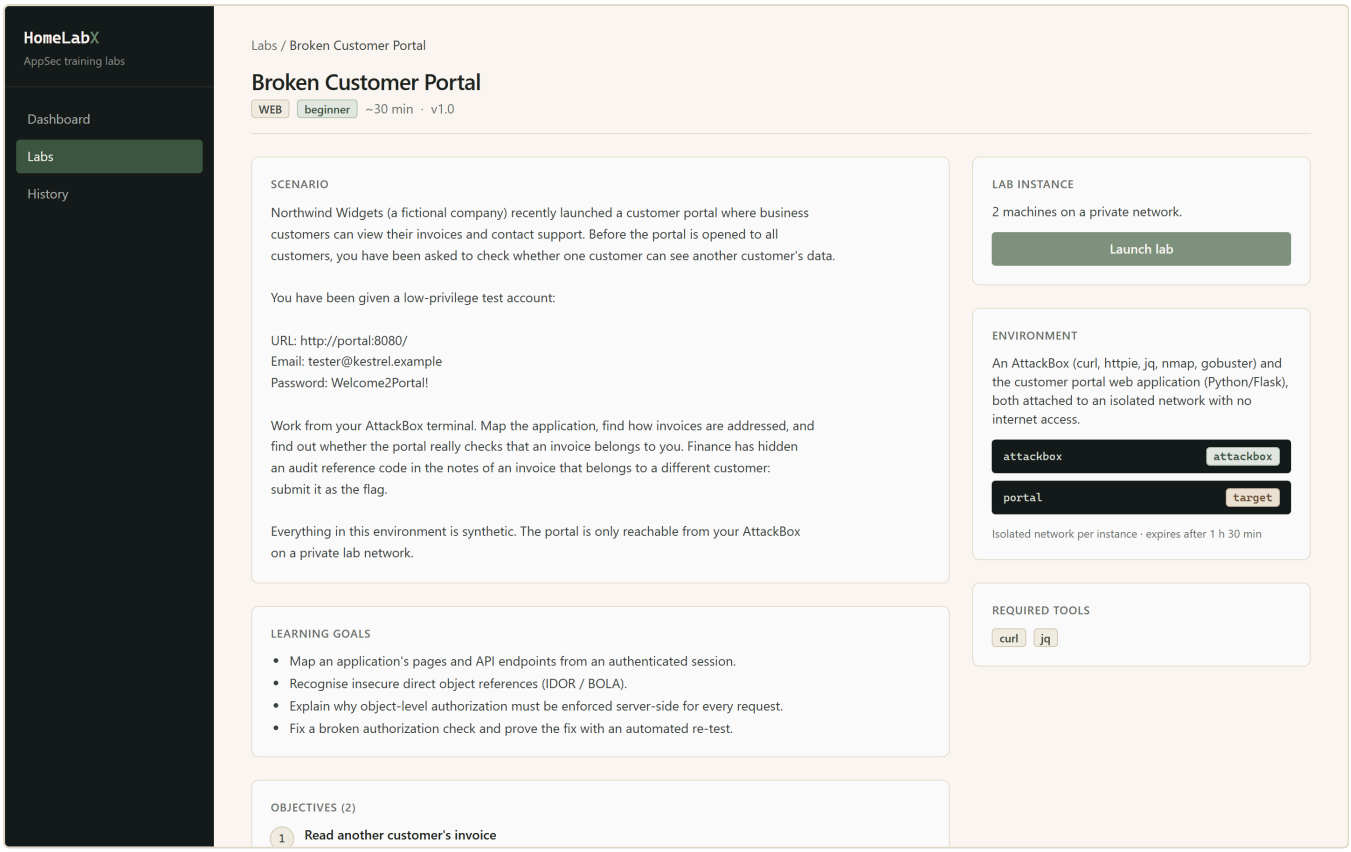


Figure 1.1 - A lab page: scenario, learning goals, objectives, hints and the launch panel. All content comes from the lab's `lab.yaml`.

The lab catalogue

The catalogue holds 13 labs, grouped into the categories the sidebar filters by:

Category	Lab	Vulnerability class
Web	broken-customer-portal	IDOR / BOLA (ships Exploit → Fix → Verify, chapter 8)
	meridian-library	SQL injection (UNION-based extraction)
	larkspur-guestbook	Stored cross-site scripting (XSS)
	nimbus-preview	Server-side request forgery (SSRF)
	aventine-docs	Path traversal
	cobalt-crm	File upload / XXE
API	orbit-api	BOLA / IDOR on a JSON API
	helios-jwt	JWT weaknesses (alg confusion, weak secret)
	atlas-api	Mass assignment
Auth	beacon-auth	Password reset / session flaws

DevSecOps	forge-ci	Secrets committed to a CI pipeline
	yamlforge	Vulnerable dependency
	sentinel-ops	Command injection / container security

Twelve of the thirteen use the browser desktop AttackBox (chapter 7.7); `meridian-library` keeps the terminal AttackBox so the Playwright suite can script its exploit.

Launching creates the lab's machines. The AttackBox opens in a workspace page with the terminal on the left and objectives, the flag form and the target list on the right:

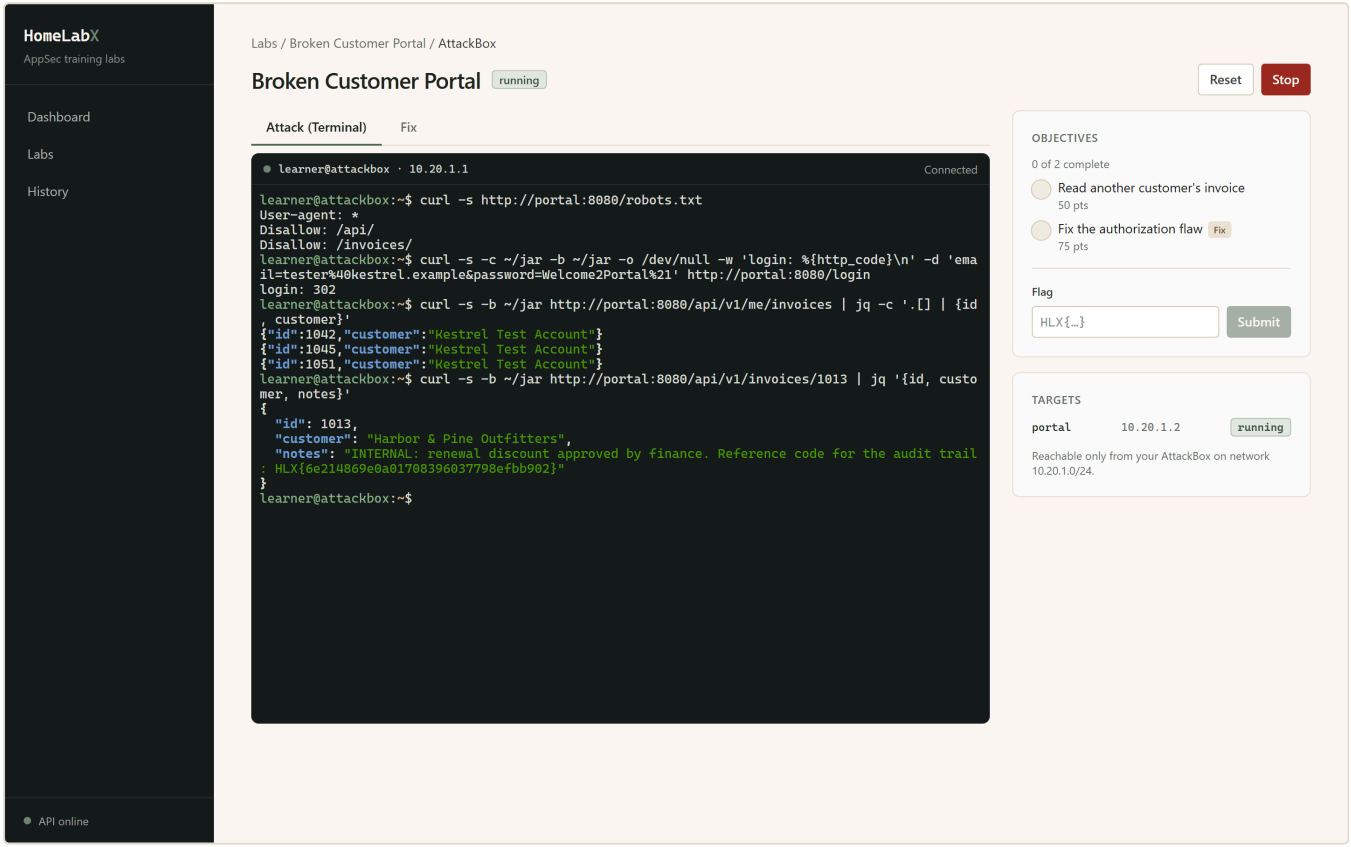


Figure 1.2 - The AttackBox workspace. Everything in the dark panel is a real bash shell inside a container on the lab's private network: here the learner logs in to the target with `curl` and reads another customer's invoice (the lab's vulnerability).

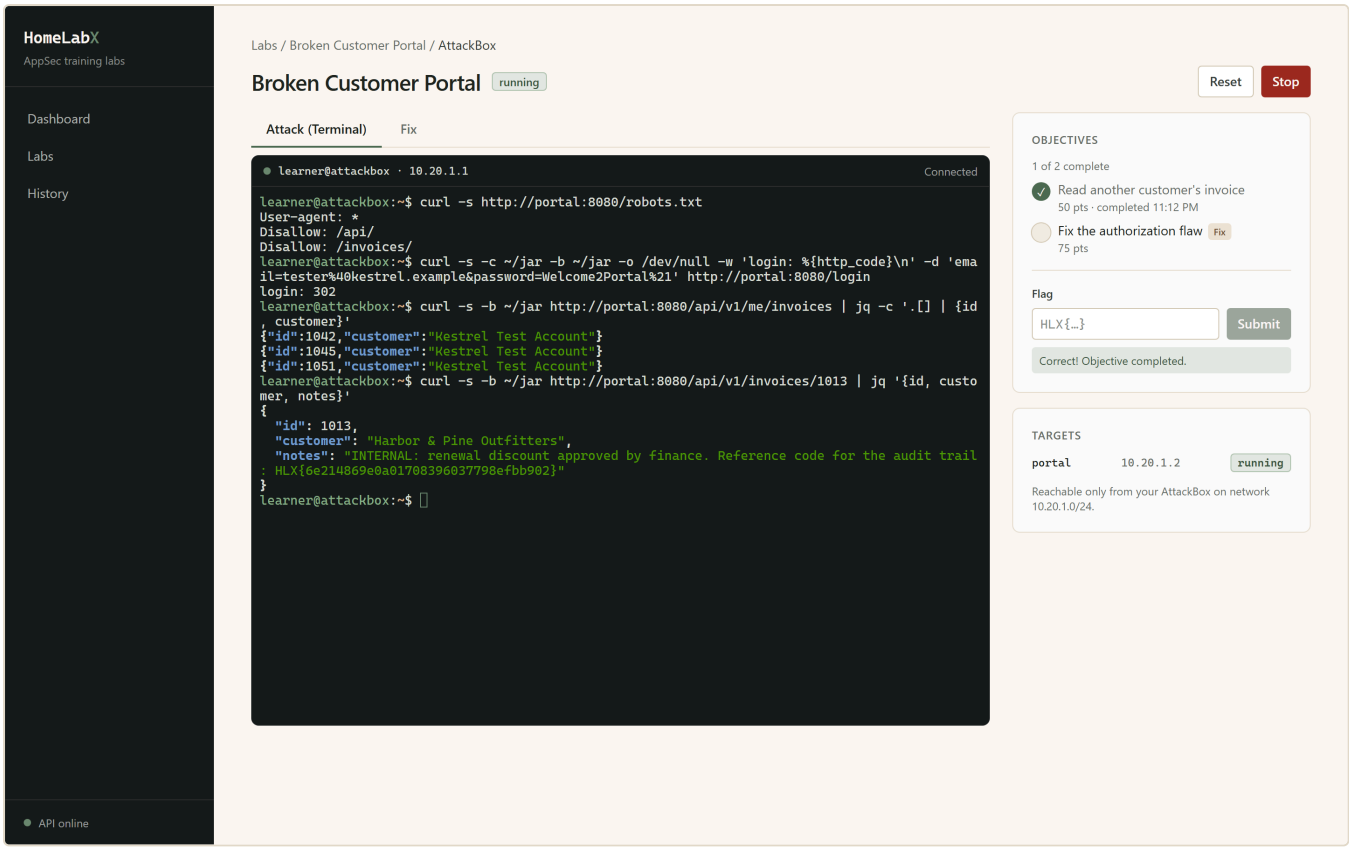


Figure 1.3 - Submitting the recovered flag completes the objective. Progress is stored in PostgreSQL and survives resets and restarts.

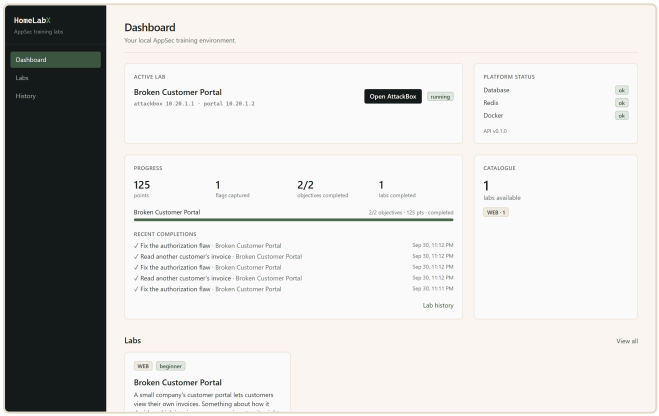


Figure 1.4 - Dashboard: active lab, platform health, progress.

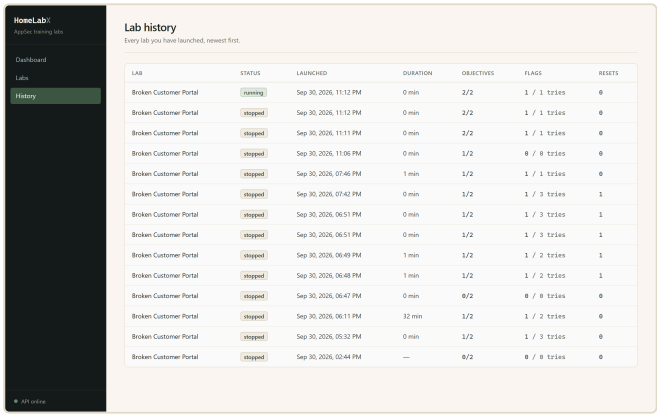


Figure 1.5 - History of every lab instance launched.

2 Architecture at a glance

HomeLabX is split into two worlds that are treated completely differently:

- The **control plane** is the trusted platform: web UI, API, database, cache and the gateway to Docker. It is defined in a Docker Compose file and runs permanently.
- The **data plane** is the labs: AttackBoxes and vulnerable targets. It is created and destroyed at runtime by the backend and is treated as **hostile**: the learner has a shell on the AttackBox, and the targets are meant to be compromised.

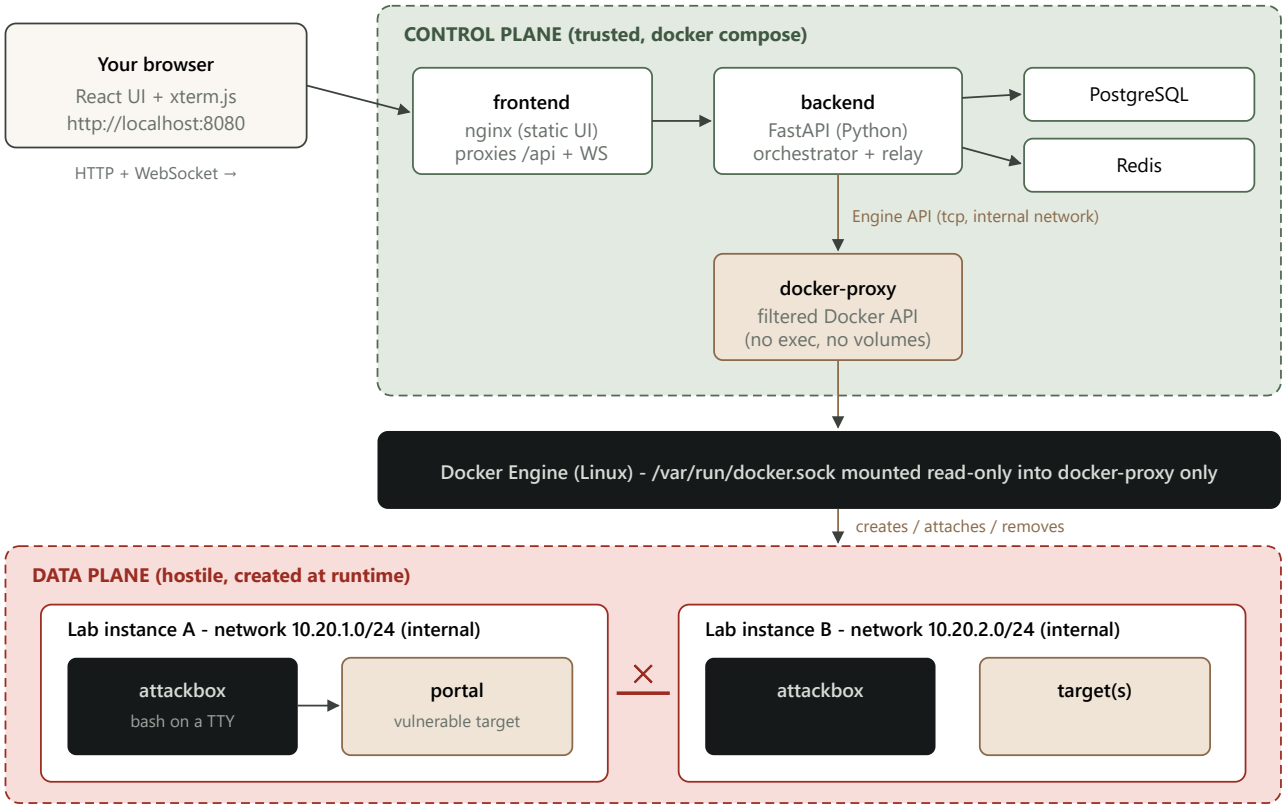


Figure 2.1 - Components and trust boundaries. Arrows show who initiates connections; nothing in the data plane can initiate a connection into the control plane, to the host, to the internet or to another lab.

What each component does

Component	Responsibility
frontend	Serves the React single-page app. In the production stack an nginx server also forwards `/api` requests and the terminal WebSocket to the backend, so the browser only ever talks to one origin.
backend	The FastAPI application: REST API, lab catalogue, **orchestrator** (creates/destroys labs), terminal relay, flag checking, progress, background reaper (expiry and cleanup).
PostgreSQL	Durable state: labs, lab instances, machines, events, flag submissions, objective completions. Schema managed by Alembic migrations.
Redis	Available for short-lived coordination; health-checked. Durable progress never lives in Redis.
docker-proxy	The only container with the Docker socket. It exposes a filtered Docker Engine API (containers, networks, images) to the backend on a private internal network, and rejects everything else (exec, volumes, image builds, swarm, secrets, system).
Lab machines	Created per lab instance from the lab definition, hardened (below), attached only to that instance's private network, and removed on stop, reset or expiry.

3 Technology stack

Layer	Technology	Role in HomeLabX and why it was chosen

UI	React 19, TypeScript, Vite	Component-based UI with static typing; Vite gives instant dev reloads and an optimised production build. The AttackBox page is lazy-loaded so the main bundle stays small.
UI data	TanStack Query	Server-state caching and polling (e.g. lab status refreshes quickly while a lab is provisioning). All API calls go through one small client with a uniform error type.
Routing / styling	React Router, Tailwind CSS	Pages for dashboard, catalogue, lab, AttackBox workspace, history; a restrained "Japandi" palette with dark terminal areas.
Terminal	xterm.js (+ fit addon)	A full terminal emulator in the browser (the same one used by VS Code): interprets ANSI escape sequences, cursor movement, colours, full-screen programs such as `vim` or `top`.
API	Python 3.13, FastAPI, Pydantic	Async HTTP + WebSocket framework with request validation from type hints. Pydantic also validates the lab definition files strictly.
Persistence	SQLAlchemy 2, Alembic, PostgreSQL 17	ORM (no hand-built SQL), versioned migrations, a proper relational database. Tests also run on SQLite for speed.
Container control	Docker SDK for Python, Docker Engine API	The orchestrator talks to Docker through its HTTP API with structured arguments; it never runs shell commands.
Docker gateway	tecnativa/docker-socket-proxy (HAProxy)	Allow-lists Docker API endpoints so even the backend cannot use exec, volumes or image builds.
Web server	nginx (unprivileged)	Production image for the UI; reverse proxy for `/api` including the WebSocket upgrade; security headers (CSP, frame denial).
Lab images	Debian (AttackBox), Python/Flask + gunicorn on Alpine (target)	The AttackBox ships curl, httpie, jq, python3, nmap, ncat/nc, dig, gobuster and wordlists; the target is a small, readable web app.
Tests	pytest, Vitest + Testing Library, Playwright	Unit, security, Docker-integration and real-browser end-to-end tests.
CI	GitHub Actions	Six jobs on every pull request; see chapter 11.
Packaging	Docker Compose, setup scripts	A development stack and a production stack from the same Dockerfiles (multi-stage builds); one-command install.

> **Deliberately not used:** Kubernetes, message queues, SSH servers, web-terminal daemons such as ttyd, and remote desktop gateways. None are needed for a single-user local platform, and each would add attack surface. The design keeps interfaces (for example the `LabRuntime` abstraction) so they can be introduced later if a real requirement appears.

4 What "the host" is: Docker, containers and the Linux VM

"The host" means different things at different layers, and the distinction matters for security. On a Windows laptop with Docker Desktop there are four layers:

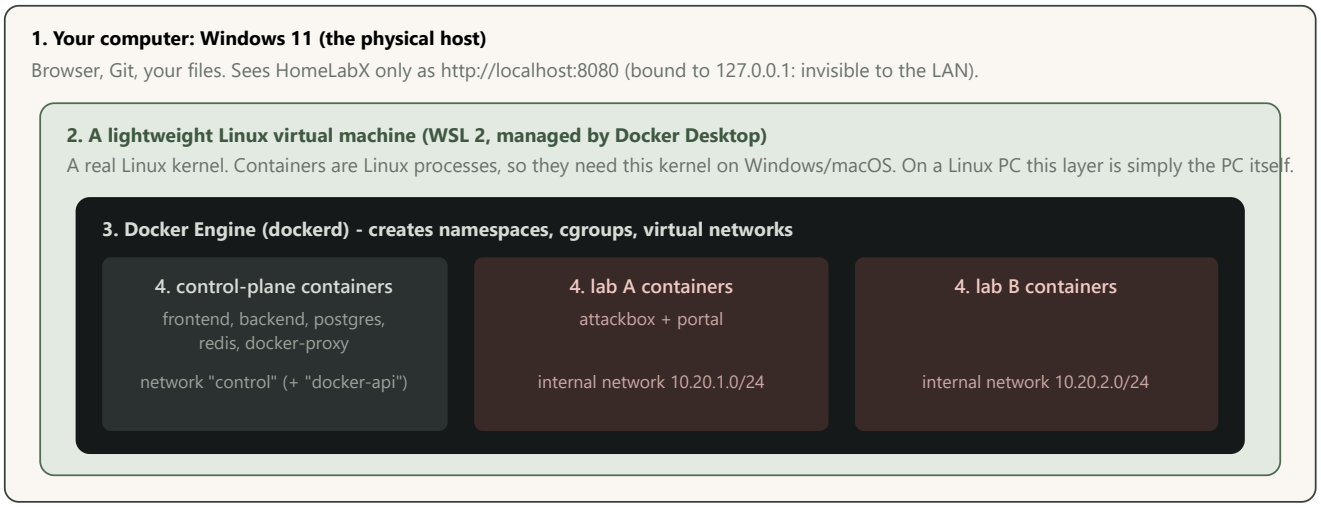


Figure 4.1 - Layers on a Windows machine. On Linux, layers 1 and 2 are the same machine.

What a container actually is

A container is **not** a virtual machine. It is an ordinary Linux process that the kernel places in its own set of *namespaces*, so it sees:

- its own filesystem (the image), mounted read-only in HomeLabX;
- its own process list (it cannot see host processes);
- its own network interfaces, attached to one Docker network;
- its own hostname and users.

Control groups (cgroups) cap how much CPU, memory and how many processes it may use. *Capabilities* split root's powers into pieces that can be removed individually. HomeLabX removes **all** of them from lab machines. All containers share the one Linux kernel, which is why the platform adds several independent layers of restriction instead of relying on any single one.

How HomeLabX configures every lab machine

Setting	Effect
not privileged, `cap_drop: ALL`, `no-new-privileges`	No kernel capabilities, even for root inside the container; setuid binaries cannot gain privileges.
read-only root filesystem	The image cannot be modified; writable paths are small in-memory *tmpfs* mounts (`noexec,nosuid,nodev`, size-capped).
CPU, memory (no swap), PID limits	A fork bomb or memory hog is contained (verified by tests).
no volumes, no bind mounts, no devices, no published ports	No access to host files or the Docker socket; nothing reachable from outside.
only the lab's own network	See chapter 6.
labels (`homelabx.instance_id`, ...)	Every resource can be found and removed reliably.

Where the ports are

Only the UI is published, on `127.0.0.1:8080` (production stack) or `127.0.0.1:5173` plus the API on `127.0.0.1:8000` (development stack). Binding to 127.0.0.1 means other devices on your Wi-Fi cannot connect.

Lab machines publish nothing at all: their services are reachable only from inside their own lab network.

5 From lab definition to running lab

Labs as code

A lab is a directory with a `lab.yaml` file (plus the source of its target images). The file describes the scenario, objectives, hints, machines, resources and flags:

```
id: broken-customer-portal
category: web        difficulty: beginner
machines:
  - name: attackbox   role: attackbox   image: homelabx/attackbox:latest
    writable_paths: ["/tmp", "/home/learner"]
  - name: portal      role: target      image: homelabx/broken-customer-portal:latest
    resources: { cpus: 0.5, memory_mb: 256, pids_limit: 128 }
    healthcheck: { test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1:8080/health"] }
objectives: [ { id: read-foreign-invoice, title: "Read another customer's invoice" } ]
flags:      [ { id: idor-flag, objective_id: read-foreign-invoice, value_env: PORTAL_FLAG, points:
50 } ]
```

The file is treated as **untrusted input**. It is parsed with a hardened YAML loader (no object tags, no alias bombs, size limits) and validated by a strict Pydantic schema. Unknown fields are rejected, so there is simply no way to ask for `privileged`, volumes, host networking or extra capabilities. Valid labs are imported into the database at startup and shown in the catalogue.

Provisioning, step by step

1. **Launch request** (`POST /api/labs/{id}/instances`): the API checks the lab exists and the active-instance limit, records a new *instance* with status `provisioning`, and returns immediately (HTTP 202). The slow work runs in a background task.
2. **Images** must already exist locally. They are never pulled from the internet at launch time, so a lab file can't smuggle in an arbitrary image.
3. **Flags**: a fresh random value such as `HLX{9f3c…}` (128 random bits) is generated for every flag. Only its SHA-256 hash is stored; the value is injected as an environment variable into the target, never into the AttackBox.
4. **Network**: a new internal bridge network with a free `/24` subnet from `10.20.0.0/16`.
5. **Machines**: one container per machine, hardened as in chapter 4, with the machine name as its DNS name (the AttackBox reaches the target as `portal`). The AttackBox also gets a TTY.
6. **Health**: the orchestrator waits for Docker health checks (bounded by a timeout).
7. **Running**: status becomes `running` and an expiry time is set (90 minutes for the sample lab).

If *anything* fails, the orchestrator finds every resource carrying the instance's labels, removes it, and marks the instance `failed` with a readable reason. Teardown is driven by labels, so it is safe to retry.

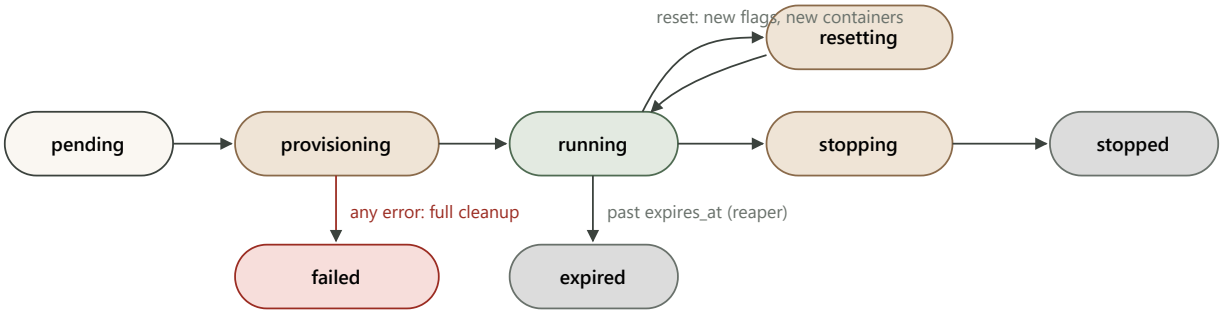


Figure 5.1 - Lifecycle of a lab instance. `stopped` and `expired` are final; launching again creates a new instance. A failed instance can be reset (retry) or stopped.

Reset, expiry and the reaper

Reset destroys every container and the network, increments the instance's *generation*, generates new flags and provisions from scratch. Anything the learner (or an exploit) changed is gone. A background **reaper** runs every 30 seconds: it expires instances past their deadline and removes any labelled resource that belongs to no active instance (for example after a crash). At startup, operations interrupted by a restart are cleaned up.

6 Networking and isolation

Each lab instance gets its **own Docker bridge network**, created with three important options:

Option	What it does
`internal: true`	Docker installs no route and no NAT out of the network: no internet, no local network.
`inhibit_ipv4`	The host side of the bridge gets no IP address, so containers have no gateway at all and cannot reach services listening on the Docker host.
own subnet (`10.20.N.0/24`)	Allocated per instance; machines reach each other by name through Docker's internal DNS (`portal` , `attackbox`).

Before attaching any container, the runtime double-checks that the target network is internal and carries HomeLabX labels. It refuses `host` , `bridge` , `none` and `container:` network modes outright.

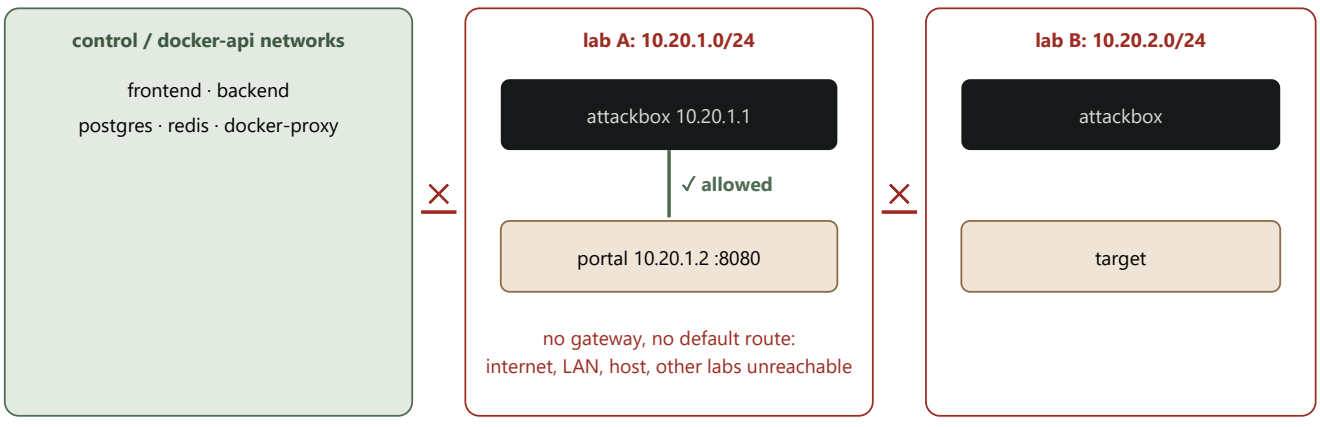


Figure 6.1 - Inside a lab, machines talk freely; across boundaries nothing is routable.

These properties are verified by automated tests that run commands inside real lab containers. The tests try to reach the internet (`1.1.1.1`), the Docker host, the host's LAN address, the Docker default gateways, every control-plane container (by IP and by name), the Docker proxy port, and the other lab's machines. All of these attempts must fail, and the lab's own target must stay reachable.

> **Accepted limitation:** inside *one* lab, every machine can reach every port of the others. Filtering per port would need extra network privileges. Everything inside one instance is the learner's own exercise, so this is documented as an accepted risk rather than hidden.

7 How the browser terminal works

The terminal in the web page is a real Linux shell running inside the AttackBox container. This chapter follows a single keystroke from your keyboard to `bash` and back.

7.1 Background: what a TTY is

Interactive programs such as `bash` expect to be connected to a **terminal** (historically a teletype, hence "TTY"). Today that terminal is virtual: a **pseudo-terminal (PTY)** made of two ends. The program uses one end as its keyboard and screen. Whoever holds the other end sends keystrokes in and receives output. The PTY driver in the kernel adds the familiar behaviour: echoing typed characters, line editing, `Ctrl-C` sending an interrupt, and knowing the window size so full-screen programs can draw.

Output is a stream of bytes that includes **ANSI escape sequences** ("move cursor", "set colour green", "clear screen"). Something must interpret those sequences and draw characters on a grid. In HomeLabX that something is **xterm.js** in your browser.

7.2 Inside the AttackBox

When the orchestrator creates the AttackBox it sets `tty: true` and `stdin_open: true`, so Docker allocates a PTY for the container's main process. The process tree is:

```
docker-init          # PID 1: a tiny init that reaps zombie processes and forwards signals
 +- attackbox-shell  # prints the banner and target list, then loops forever:
     +- bash --login # your shell, running as the unprivileged user "learner" (uid 1000)
         +- curl / nmap / python3  ...
```

If you type `exit`, bash ends and `attackbox-shell` starts a fresh one, so the terminal never "dies". The home directory is an in-memory tmpfs, so every reset gives a clean home.

7.3 Why Docker *attach* (and not exec, SSH or a terminal server)

Option	Why it was rejected or chosen
SSH server in the AttackBox	Needs a network path from the platform (or the browser) into the lab, keys or passwords, and an exposed port. Rejected.
Web-terminal daemon (ttyd, gotty)	Same network-path problem, and it's another service inside a hostile container. Rejected.
`docker exec`	Would require enabling `exec` on the Docker proxy, and that allows running commands in *any* container, including the database or the backend. Rejected.
`docker attach`	**Chosen.** It connects to the PTY of the process the container was *created* with, and nothing else. It goes through the Docker API, so no network connection into the lab exists at all. The proxy already allows it.

7.4 Connecting: tickets and the WebSocket

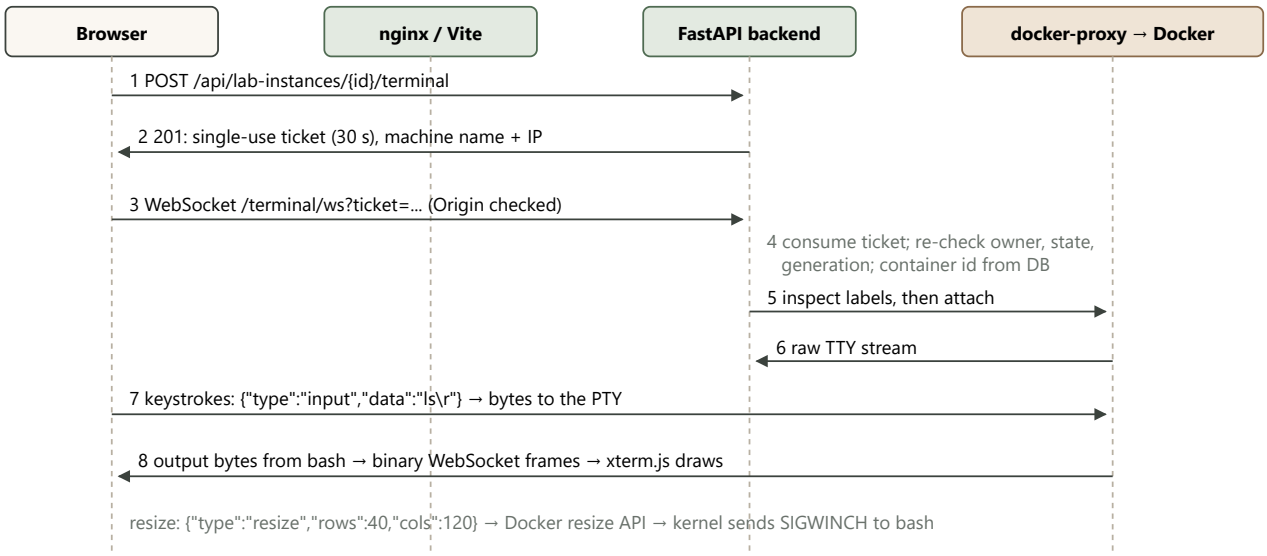


Figure 7.1 - Opening a terminal session. Steps 7 and 8 repeat for as long as the session lasts.

Step 1-2: the ticket. The browser first asks the REST API for a ticket. The API checks that the instance belongs to the user and is running, then returns 32 random bytes, valid for 30 seconds and usable **once**. The ticket is bound to that instance, user, reset generation and machine.

Step 3: the WebSocket. A WebSocket is a long-lived, two-way connection that starts as an HTTP request and is then "upgraded". Its handshake must come from an allowed `Origin`. Otherwise a malicious website you happen to visit could open a terminal on your lab through your browser (cross-site WebSocket hijacking). nginx (production) or Vite (development) forwards the upgrade to the backend.

Steps 4-6: attach, safely. The backend never takes a container id from the browser. It looks up the AttackBox's container id in the database and inspects the container. It only attaches if the container carries the expected labels (this deployment, this instance, this generation, `role=attackbox`), was created with a TTY, and is running. Even a tampered database row could therefore not open a shell in a target or another lab (tested). The attach request goes through the Docker proxy, where HAProxy passes the HTTP "upgrade" to Docker, which returns the raw PTY stream.

Steps 7-8: the relay. For each session the backend runs two small loops: *browser → PTY* (validates each JSON message: only `input` of at most 16 KiB and `resize` within bounds; anything else closes the session with code 4400) and *PTY → browser* (reads output in a worker thread and forwards it as binary frames). On connect it sends `Ctrl-L` so bash redraws the prompt for the new viewer. Docker's default detach shortcut (`Ctrl-P Ctrl-Q`) is replaced so that `Ctrl-P` works normally in the shell.

7.5 In the browser

xterm.js renders the character grid and turns your keystrokes (including `Tab`, arrows, `Ctrl` combinations) into the bytes a terminal would send. The *fit* addon measures the panel and reports rows and columns, which the backend forwards to Docker's resize API. The kernel then notifies bash (`SIGWINCH`), so line wrapping and full-screen programs adapt. When the lab is reset or stopped, the container disappears, the stream ends and the page shows *Disconnected* with a **Reconnect** button. After a reset the page automatically connects to the new AttackBox.

7.6 Limits and auditing

- At most 3 simultaneous terminals per lab; sessions end when the lab expires (at most 8 h).
- Every open and close is recorded as a lab event with duration and byte counts. Keystrokes are never logged.
- All browser tabs attached to one AttackBox share one shell, like `screen -x`.

7.7 The desktop AttackBox (noVNC)

Some labs (file upload, SSRF, JWT tooling, path traversal, and more) are more natural to attack with a browser, a proxy and a file manager than with `curl` alone. A lab opts in with `desktop: true`, which gives it `homelabx/attackbox-desktop` instead of the terminal image: Debian + XFCE, Firefox, a file manager, a text editor, **mitmproxy**, and the same CLI tools as the terminal AttackBox, served over VNC by TigerVNC + websockify + noVNC on port 6080 inside the container.

The platform still never publishes that port. The backend and every desktop AttackBox share one extra **internal, no-egress** network (`desktop-gw`); targets never join it. The frontend's Desktop tab lazy-loads noVNC's `RFB` client (so terminal-only labs never pay for it) and opens a WebSocket to the backend, which relays it to `ws://dsk-<instance>:6080/websockify` using the same single-use-ticket-plus-Origin-guard pattern as the terminal (7.4). A readiness health check gates the instance's `running` state on noVNC actually serving, so the UI never shows a desktop before it can paint.

8 The first lab: Broken Customer Portal

Scenario. Northwind Widgets, a fictional company, launched a customer portal where business customers view invoices. The learner receives a low-privilege test account and must find out whether one customer can see another customer's data. All data is synthetic: fictional companies, `.example` e-mail addresses, generated invoices.

The vulnerability: broken object-level authorization (IDOR / BOLA). The invoice page and the JSON API check that you are *logged in*, but not that the invoice is *yours*. Invoice numbers are sequential, so changing `1042` to `1013` in the URL shows another company's invoice:

```
@app.get("/api/v1/invoices/<int:invoice_id>")
@login_required                                  # authentication: yes
def api_invoice(invoice_id):
    invoice = INVOICES.get(invoice_id)            # VULNERABLE: no ownership check
    if invoice is None:
        return jsonify(error="invoice not found"), 404
    return jsonify(public_invoice(invoice))
```

The fix is one line of authorization logic: return 404 unless `invoice["customer_id"] == session["customer_id"]`. Everything else in the portal is written correctly on purpose (the "my invoices" listing filters by owner, templates escape output, there is no open redirect), so the lesson stays focused on a single concept.

The flag. The per-instance random flag (from the `PORTAL_FLAG` environment variable) is written into the notes of invoice 1013, which belongs to "Harbor & Pine Outfitters". The learner enumerates invoice ids from the AttackBox, finds the note and submits the flag.

Clean resets. Customers, invoices, support tickets and sessions exist only in memory, and the session-signing key is random at every start. A reset (a brand-new container) therefore always returns the portal to its seeded

state, invalidates old sessions and carries a new flag. The Docker integration test for this lab plays the whole scenario through the real terminal. It then leaves some state behind, resets, and verifies all of this.

Target hardening	Value
Image	`python:3.13-alpine`, Flask 3.1.3 + gunicorn (1 worker, 8 threads), non-root user
Limits	0.5 CPU, 256 MB RAM, 128 processes, read-only rootfs, 64 MB tmpfs `/tmp`
Reachability	port 8080, only from its own lab network

Exploit → Fix → Verify

This lab is also the platform's first **remediation** lab, the feature that sets HomeLabX apart from a collection of vulnerable machines: after exploiting the flaw, the learner fixes it and the platform proves the fix. The portal's authorization decision lives in one file, `policy.py` (`can_view_invoice(customer, invoice)`), which as shipped returns `True` for any logged-in customer.

1. In the **Fix** tab the learner edits `policy.py` and submits it.
2. The platform recreates the `portal` container with the edited file delivered as a base64 environment variable (same image, flags and hardening; no exec, no file copy). If the code does not start, the original is restored and the error is shown.
3. A one-shot, hardened **verifier** container runs on the lab network: it re-runs the original IDOR exploit and checks that a customer can still see their own invoices, printing a JSON result.
4. The objective completes only when the exploit is blocked *and* legitimate access still works, so a fix that blocks everything fails.

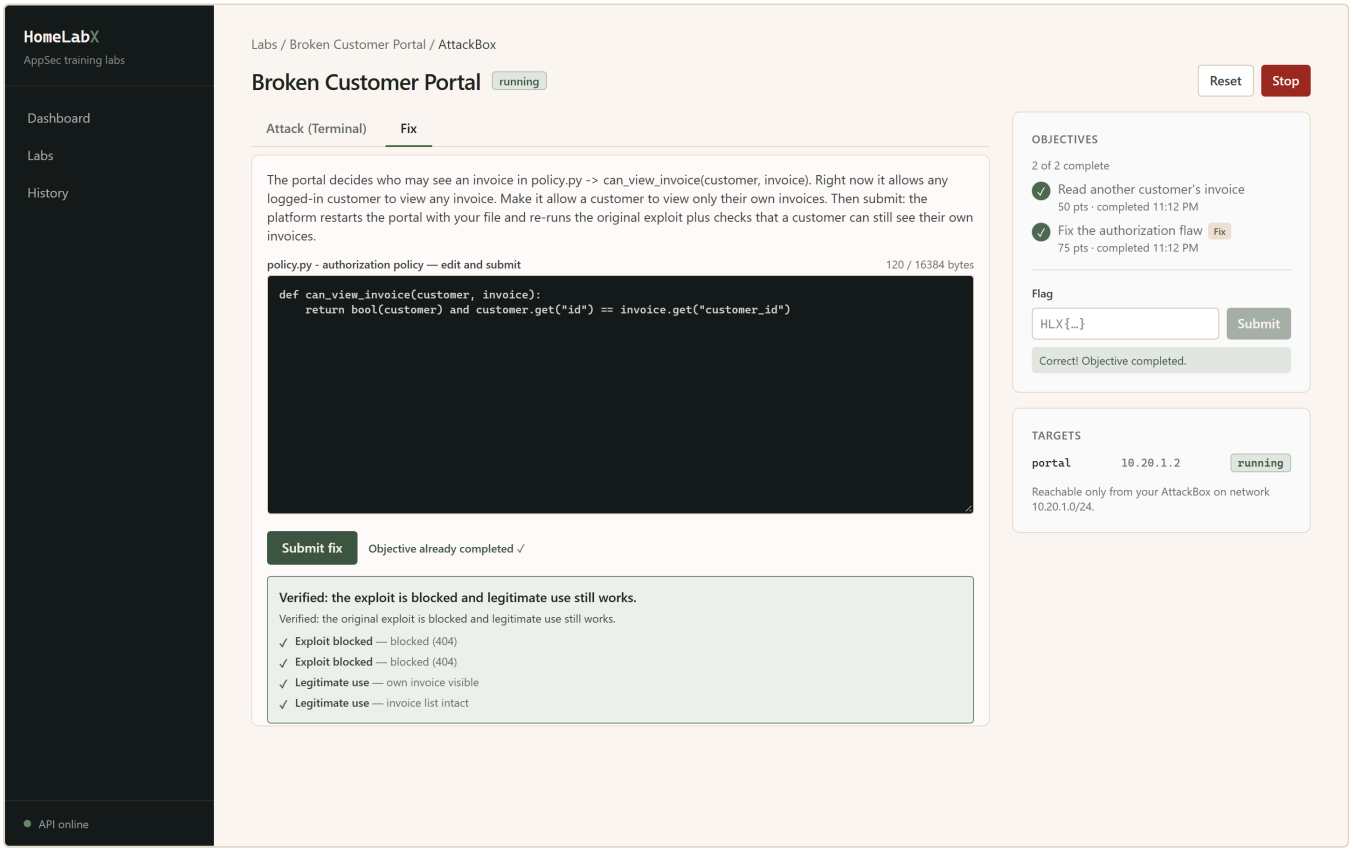


Figure 8.1 - Fix mode: the learner's corrected `policy.py`, the verifier's per-check results (exploit blocked, legitimate use intact), and the completed "Fix" objective. The learner's code only ever runs inside the already-isolated target container.

9 Flags, objectives and progress

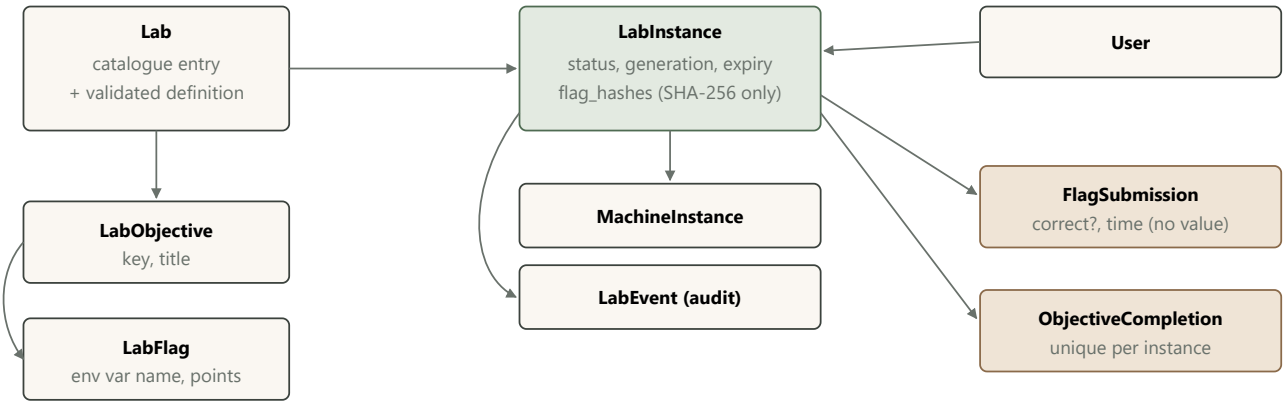


Figure 9.1 - Data model (simplified). Arrows point from "has many" to the child.

Submission rules

- A flag is accepted only while the instance is *running*. It is hashed and compared in constant time against the hashes of the **current generation**, so a flag from before a reset, from another instance or from another lab never matches.
- The submitted value is never stored or logged; only the outcome is.
- A correct flag completes its objective once per instance. Resubmitting returns `already_captured` and changes nothing.
- At most 10 wrong submissions per instance per minute (HTTP 429 after that).
- Every query is scoped through the instance's owner. Another user's instance is indistinguishable from a non-existent one (HTTP 404). This is multi-user-ready even though the MVP has a single local user.

The dashboard shows each objective's *first* completion across all of the learner's instances, so progress survives stop, reset, expiry and platform restarts.

10 Security model

Threat model

Assume the targets are fully compromised and the learner (or code running in any lab) actively tries to escape: attack the host, other labs, the platform, or the local network. The host and the control plane must remain protected.

Layers of defence

Area	Controls
Container	no privileges or capabilities, no-new-privileges, read-only rootfs, noexec tmpfs, CPU/memory/PID limits, no mounts/devices/ports, non-root users where practical
Network	per-instance internal network without a gateway; runtime refuses non-lab networks
Docker access	only the backend, only through the filtering proxy (no exec, volumes, builds); socket never in any lab; images never pulled at launch

Lab definitions	strict schema (unknown fields rejected), hardened YAML loader, only a small set of re-addable capabilities; the runtime re-validates every container spec
API	validated IDs and bodies, ORM-only database access, no shell anywhere, generic 500 errors, no secrets or runtime internals in responses
Browser	**Origin allow-list** for state-changing requests and all WebSockets (blocks cross-site request forgery and WebSocket hijacking); **Host allow-list** (blocks DNS rebinding); CSP and frame denial in nginx
Lifecycle	label-driven cleanup, expiry, orphan reaping, startup recovery
Supply chain	pip-audit and npm audit in CI; this found and fixed an outdated Flask in the lab image

> **Why the Origin/Host guards matter.** The MVP has no login: anyone who can reach `127.0.0.1` is the user. Websites you visit can make your browser send requests to `localhost`. Without these guards, a malicious page could launch labs, submit flags or open your AttackBox terminal. Browsers always send an `Origin` header on cross-site POSTs and WebSocket handshakes, and the API rejects unknown origins. This gap was found during the security audit and is covered by tests.

Evidence

`SECURITY.md` contains a line-by-line audit: every requirement, the control that implements it, and the automated test that proves it. Examples of tests that run against real containers:

- the AttackBox cannot reach lab B, the internet, the host or any control-plane service;
- a target (even running as root) has zero capabilities and cannot reach Docker;
- a fork bomb hits the process limit and the shell survives;
- tmpfs refuses writes past its size and won't execute dropped binaries;
- a terminal ticket works once, only for its instance and only from the UI origin;
- the real background reaper expires a lab and removes all of its resources.

Accepted risks (documented)

> **1. The backend is trusted with Docker.** The proxy cannot inspect request bodies, so a compromised backend could create a dangerous container. It is reachable only locally and never from labs, and it validates every spec itself.
> **2. No per-port filtering inside one lab** (chapter 6).
> **3. No authentication** in the single-user MVP. It must be added before any multi-user or network-exposed deployment.

11 Testing and continuous integration

Suite	Tests	What it proves

Backend unit + security	~160	Lab schema and hostile YAML, lifecycle state machine, orchestration with an in-memory Docker fake (including failure injection), terminal and desktop (noVNC) protocol and tickets, remediation (exploit/fix/verify), flags and progress, cross-user and cross-instance authorization, Origin/Host guards, static scans (no shell or subprocess, no string-built SQL, Compose rules, no committed secrets, synthetic lab data, CI cannot fail softly).
Docker integration	~46	Real labs on a real engine, across all 13 lab definitions: hardening via container inspection, isolation probes, the AttackBox terminal and desktop over a real WebSocket, the exploit/fix/verify flow, reset/stop/expiry cleanup, and no leftover resources after any test.
Frontend	~42	Components and pages with mocked APIs; the terminal WebSocket client and the Fix-mode panel with fake sockets/responses.
End-to-end (Playwright)	2	A real browser against the running platform: dashboard → lab → launch → AttackBox → exploit in the terminal → wrong and right flag → completion → reset (fresh box, old flag rejected) → progress and history → stop; plus a cross-site request check.

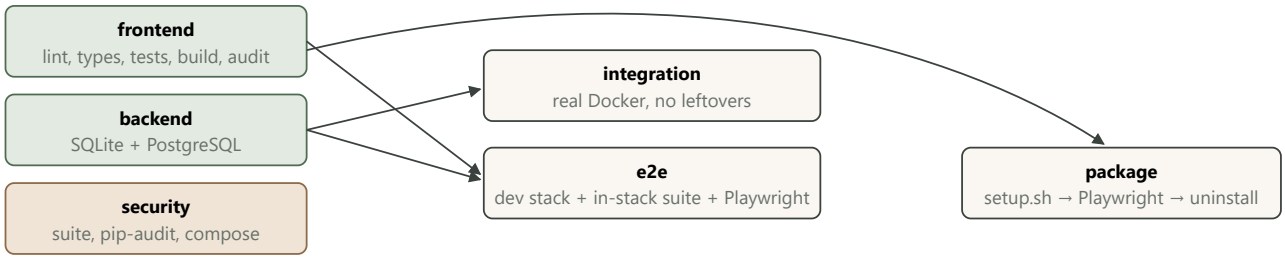


Figure 11.1 - GitHub Actions jobs. All must pass; a static test fails the build if anyone adds `continue-on-error` or removes a required step.

Fresh-clone verification. `fresh-clone-check.sh` clones the committed code into a temporary folder and runs a second, fully separate stack next to the developer's own (different ports, database and deployment label). It builds everything from scratch, runs the test suites and Playwright, and removes it all again. Its `--prod` mode exercises exactly what a new user runs (`setup.sh`). Running it caught four real problems that normal test runs could not: an outdated dependency with a published advisory, tests that silently depended on the developer's configuration, a shell-script bug that would have broken installation on every Linux and macOS machine, and a false-positive port check.

12 Packaging: running it on another machine

What someone needs

Only **Docker** (Docker Desktop on Windows/macOS, Docker Engine + Compose on Linux) and **Git**, about 4 GB of free RAM for Docker and ~3 GB of disk. Internet access is needed only during the first build.

```
git clone https://github.com/ParitV/HomeLab.git
cd HomeLab
./infrastructure/scripts/setup.sh          # Windows PowerShell: .\infrastructure\scripts\setup.ps1
#  → HomeLabX is running: http://localhost:8080
./infrastructure/scripts/setup.sh --down       # stop (progress kept)
./infrastructure/scripts/setup.sh --uninstall  # remove containers, data and lab resources
```

What the setup script does

1. Checks that Docker and Compose are installed and running.
2. Creates `.env` from `.env.example` with a **randomly generated** database password (only on first run; the file is git-ignored).
3. Builds everything locally: the platform images and the lab images (AttackBox, targets). On first run this downloads base images and packages.
4. Starts the production stack (`compose.prod.yml`) and waits for `/api/health` to report all checks OK.
5. `--down` stops running labs cleanly first (so nothing keeps running unnoticed); `--uninstall` also deletes the database volume and any lab resources.

Two stacks from the same Dockerfiles

	Development (`docker-compose.yml`)	Production (`compose.prod.yml`)
Who	people changing the code	people using HomeLabX
UI	Vite dev server, hot reload, :5173	optimised static build behind nginx, :8080
API	auto-reload, published on 127.0.0.1:8000	not published: reachable only through nginx
Code	mounted from the checkout	baked into images (labs mounted read-only)
Image targets	`dev`	`runtime` (backend), `prod` (frontend)
Secrets	local placeholders	random password generated by setup

Both stacks can run side by side: they use different Compose projects, databases and deployment labels, so neither ever cleans up the other's labs. CI's `package` job runs the production path from a clean checkout on every pull request.

Distribution options

Option	Trade-off
Clone + setup script (current)	Simple and transparent: users build from source they can read. First build takes several minutes and needs internet.
GitHub Release `v0.1.0`	A stable, tagged version with install notes and this report attached.

Prebuilt images on GHCR (future)	No local build: `docker compose pull` and start. Needs a publishing pipeline, image signing and versioning; users must trust the published images.

> **For anyone installing it:** HomeLabX contains deliberately vulnerable software. It is safe by design only as long as the defaults are kept: never publish the API or lab containers to your network, and run it on a machine you control.

13 Roadmap

Phase	Content
0-9 (done)	Foundation, backend, dashboard, lab definitions, orchestration, AttackBox and terminal, first lab, progress, security hardening, E2E and CI, packaging. Tagged as **v0.1.0**.
5C - Desktop AttackBox (done)	A second AttackBox flavour (chapter 7.7): XFCE over noVNC, relayed by the backend through an internal gateway network. 12 of 13 labs now default to it; the terminal AttackBox remains available and is what the E2E suite drives.
10 - AppSec catalogue (done)	Twelve additional labs, each with its own scenario, synthetic data, flag, reset and tests (chapter 1 table): SQL injection, XSS, SSRF, path traversal, file upload/XXE, API BOLA, JWT weaknesses, mass assignment, password reset/session flaws, and three DevSecOps labs (secrets, vulnerable dependency, container/command injection).
11 - Exploit → Fix → Verify (done)	The project's differentiator, implemented on the Broken Customer Portal (chapter 8): exploit the flaw, edit the vulnerable source, and the platform recreates the target with the fix and re-runs the original exploit plus regression checks to verify it. Built on the existing lab lifecycle.
12 - Scaling (proposed, not implemented)	A documented, staged upgrade path in `docs/SCALING.md`: authentication & roles → host concurrency & quotas → multiple backend workers → separate Docker hosts (TLS) → Kubernetes only if a real need appears. Each step has a concrete trigger; the local Docker setup stays the default for this portfolio project.

A Repository map and glossary

```
apps/backend/app/          FastAPI application
  api/                     HTTP + WebSocket endpoints (labs, instances, terminal, desktop,
progress)
  services/labs/           lab.yaml schema, hardened loader, catalogue import
  services/orchestration/  runtime abstraction, Docker runtime, orchestrator, lifecycle
  services/terminal.py     tickets and terminal sessions          services/desktop.py   noVNC relay
  services/remediation.py  exploit → fix → verify flow
  services/progress/       flag checking, objectives, progress, history
  core/security.py         Origin guard (CSRF / WebSocket hijacking)
apps/backend/tests/        unit/, security/, integration/
apps/frontend/src/         React app (features/: dashboard, labs, attackbox, progress)
labs/web/, labs/api/, labs/auth/, labs/devsecops/   13 labs, each lab.yaml + target/ (+ verifier/
if remediation)
infrastructure/docker/attackbox/, attackbox-desktop/   terminal and XFCE/noVNC AttackBox images
infrastructure/scripts/    setup, check, fresh-clone-check
tests/e2e/                 Playwright
docker-compose.yml         development stack      compose.prod.yml   production stack
.github/workflows/ci.yml   CI                     docs/              guides + this report
```

Term	Meaning
AttackBox	The learner's Linux workstation inside a lab, used through the browser terminal.
Target	A deliberately vulnerable machine in a lab.
Lab instance	One launched copy of a lab: its own network, containers and flags.
Generation	Counter incremented on every reset; flags and tickets from older generations are invalid.
Flag	A random secret (`HLX{ ... }`) placed in a target; submitting it proves an objective was achieved.
Control / data plane	The trusted platform vs. the hostile lab machines.
Namespace, cgroup, capability	Linux kernel features that isolate a container's view, cap its resources and split root's privileges.
TTY / PTY	(Pseudo-)terminal: the kernel device interactive programs talk to; the browser terminal is its far end.
Docker attach / exec	Connect to a container's main process / start a new process in a container.
WebSocket	A long-lived two-way connection over HTTP, used for the terminal stream.
IDOR / BOLA	Insecure direct object reference / broken object-level authorization: accessing objects by id without an ownership check.
Socket proxy	A filter in front of the Docker API that only allows listed operations.
Reaper	Background task that expires labs and removes orphaned resources.